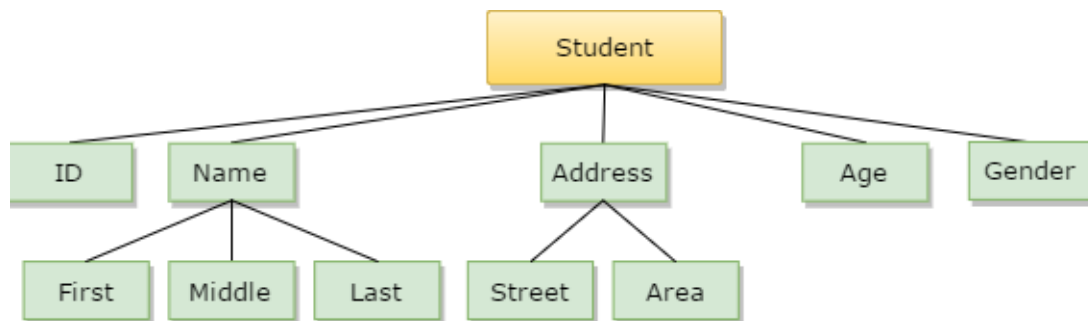


Object Oriented Programming & Data Structure

Unit 4. Data Structure

Data Structure Introduction:

In the modern world, Data and its information is an essential part, and various implementations are being made to store in different ways. Data are just a collection of facts and figures, or you can say data are values or a set of values that are in a particular format. A data item refers to a single set of values. Data items are then further categorized into sub-items, which are the group of items that are not being called a plain elementary form of items. Let us take an example where the name of the student may be divided into three sub-items, namely: first name, middle name, and last name. But the ID that is assigned to a student would typically be considered as a single item.



In the example mentioned above, such as ID, Age, Gender, First, Middle, Last, Street, Area, etc. are elementary data items, whereas (Name, Address) is group data items.

What is Data Structure?

In computer terms, a data structure is a Specific way to store and organize data in a computer's memory so that these data can be used efficiently later. Data may be arranged in many different ways, such as the logical or mathematical model for a particular organization of data is termed as a data structure. The variety of a specific data model depends on the two factors -

- Firstly, it must be loaded enough in structure to reflect the actual relationships of the data with the real-world object.
- Secondly, the formation should be simple enough so that anyone can efficiently process the data each time it is necessary.

Categories of Data Structure:

The data structure can be subdivided into major types:

- Linear Data Structure

Object Oriented Programming & Data Structure

- Non-linear Data Structure

Linear Data Structure

A data structure is said to be linear if its elements combine to form any specific order. There are two techniques of representing such linear structure within memory.

- The first way is to provide the linear relationships among all the elements represented using linear memory location. These linear structures are termed as arrays.
- The second technique is to provide a linear relationship among all the elements represented by using the concept of pointers or links. These linear structures are termed as linked lists.

The common examples of the linear data structure are:

- Arrays
- Queues
- Stacks
- Linked lists

Nonlinear Data Structure

This structure is mostly used for representing data that contains a hierarchical relationship among various elements.

Examples of Non-Linear Data Structures are listed below:

- Graphs
- the family of trees and
- table of contents

Tree: In this case, data often contain a hierarchical relationship among various elements. The data structure that reflects this relationship is termed as a rooted tree graph or a tree.

Graph: In this case, data sometimes hold a relationship between the pairs of elements, which is not necessarily following the hierarchical structure. Such a data structure is termed as a Graph.

Object Oriented Programming & Data Structure

What is Stack ?

A stack is a linear data structure in which all the insertion and deletion of data or you can say its values are done at one end only, rather than in the middle. Stacks can be implemented by using arrays of type linear.

The stack is mostly used in converting and evaluating expressions in Polish notations, i.e.:

- Infix
- Prefix
- Postfix

In case of arrays and linked lists, these two allows programmers to insert and delete elements from any place within the list, i.e., from the beginning or the end or even from the middle also. But in computer programming and development, there may arise some situations where insertion and deletion require only at one end wither at the beginning or end of the list. The stack is a linear data structure, and all the insertion and deletion of its values are done in the same end which is called the top of the stack. Let us suppose take the real-life example of a stack of plates or a pile of books etc. As the item in this form of data structure can be removed or added from the top only which means the last item to be added to the stack is the first item to be removed. So you can say that the stack follows the Last In First Out (LIFO) structure.

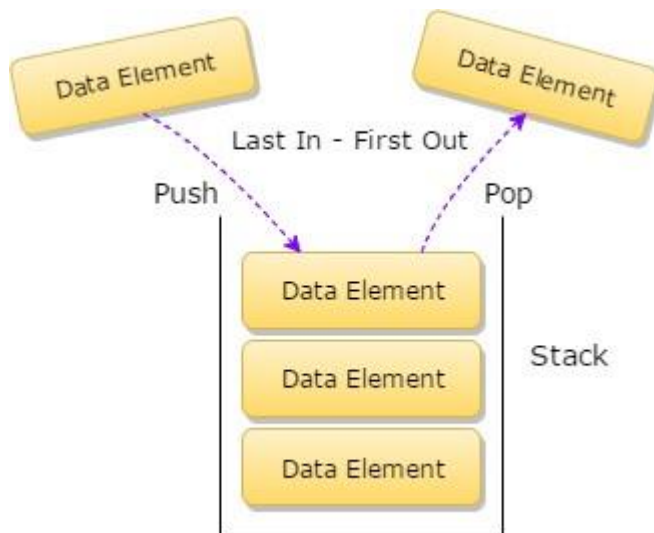
Stack as Abstract data type:

He stacks of elements of any particular type is a finite sequence of elements of that type together with the following operations:

- Initialize the stack to be empty
- Determine whether the stack is empty or not
- Check whether the stack is full or not
- If the stack is not full, add or insert a new node at the top of the stack. This operation is termed as Push Operation
- If the stack is not empty, then retrieve the node at its top

Object Oriented Programming & Data Structure

- If the stack is not empty, delete the node at its top. This operation is called as Pop operation



Stack Definition And Concept

Stack is a sub class of linear data structure in which elements can be inserted or deleted from only one end called the top of the stack. Since insertion and deletion in stack is performed only from one end, its elements are removed from stack in reverse order from the order in which they were inserted. This means that the last item added in the list is the first item to be removed. Because of this characteristic stack is called LIFO (Last In First Out) list. It is also called a 'Pile' or 'Push Down List'.

Real life example of stack is box containing books which can be opened only from one end and only one book can be removed from box at a time. Tray holder in cafeteria is also an example of a stack.

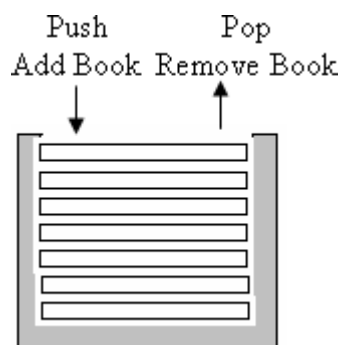


Fig. 3.5 Books Contained Within Box

Stack Operations

Operations on stack include

PUSH for inserting an element into stack.

POP for deleting an element from stack.

PEEP for getting the value of specified element from the TOP of the stack.

CHANGE for changing the value of specified element from the TOP of the stack.

Object Oriented Programming & Data Structure

An Algorithm To Insert An Element Into Stack

The main tasks in this algorithm are:

- Check for overflow condition.
- Increment TOP.
- Insert the element.

PUSH (S, TOP, ITEM)

S: An array representing stack.

TOP: indicates position of topmost element in the stack.

ITEM: Value to be inserted.

MAX: Maximum number of elements that can be inserted into stack.

Step 1: [Check for Stack Overflow]

 If $TOP \geq MAX-1$ then

 Write (“Stack Overflow”)

 Exit

 [End If Structure]

Step 2: [Increment TOP]

$TOP := TOP+1$

Step 3: [Insert Element]

$S[TOP] := ITEM$

Step 4: [Finished]

 Return

An Algorithm To Delete An Element From Stack

The main tasks in this algorithm are:

- Check for underflow condition.
- Decrement TOP.

POP (S, TOP)

S: An array representing stack.

TOP: indicates position of topmost element in the stack.

Step 1: [Check for underflow on stack]

 If $TOP < 0$ then

 Write (“Stack Underflow on POP”)

 Exit

 [End If Structure]

Step 2: [Decrement Pointer]

$TOP := TOP-1$

Step 3: [Return top element of stack]

 Return (S [TOP+1])

An Algorithm To Obtain The Value Of Ith Element From Top Of The Stack Without Deleting It

Object Oriented Programming & Data Structure

The main tasks in this algorithm are:

- Check for underflow condition.
- Retrieve the desired element.

PEEP (S, TOP, I)

S: An array representing stack.

TOP: indicates position of topmost element in the stack.

I: Position of the element to be returned from TOP of the stack.

Step 1: [Check for Stack Underflow]

 If $TOP - I + 1 \leq -1$ then

 Write (“Stack Underflow on PEEP”)

 Exit

 [End of if structure]

Step 2: [Return I^{th} value from top of stack]

 Return (S[TOP – I + 1])

An Algorithm To Change The Value Of I^{th} Element From Top Of The Stack

The main tasks in this algorithm are:

- Check for underflow condition.
- Update the desired element.

CHANGE (S, TOP, ITEM, I)

S: An array representing stack.

TOP: indicates position of top element in the stack.

I: Position of the element to be changed from TOP of the stack

ITEM: New value of I^{th} element from top of the stack.

Step 1: [Check for Stack Underflow]

 If $TOP - I + 1 \leq -1$ then

 Write (“Stack Underflow on change”)

 Exit

Step 2: [Change I^{th} element from top of Stack]

 S[TOP – I + 1] := ITEM

Step 3: [Finished]

 Exit

Representation of Stack using Arrays

The stack can be represented in memory with the use of arrays. To do this job, you need to maintain a linear array STACK, a pointer variable top which contains the top element.

Object Oriented Programming & Data Structure

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>

class stack {
    int stk[5];
    int top;

public:
    stack()
    {
        top = -1;
    }
    void push(int x)
    {
        if (top >= 4) {
            cout << "stack overflow";
            return;
        }
        top=top+1;
        stk[top] = x;
        cout << "inserted " << x;
    }
    void pop()
    {
        if (top < 0) {
            cout << "stack underflow";
            return;
        }
        cout << "deleted " << stk[top--];
    }

    void display();
    void peep();
    void change();
};

void stack::display()
{
    if (top < 0) {
        cout << " stack empty"; return;
    }
    for (int i = top; i >= 0; i--)
        cout << stk[i] << " ";
}

void stack::peep()
```

Object Oriented Programming & Data Structure

```
{    int i;
    cout<<"enter the position";
        cin>>i;
    if (top < 0) {
        cout << " stack empty"; return; }
    cout<<stk[top-i+1]; //
}

void stack::change()
{    int i,item;
    //cout<<"\n value of top is "<<top;
    cout<<"\n enter the position to enter";
        cin>>i;
    cout<<"\n Enter the item to be insert :";
    cin>>item;
    if (top < 0) {
        cout << " stack empty"; return; }
    stk[top-i+1]=item;
}

int main()
{    clrscr();
    int ch;
    stack st;
    while (1)
    {
        cout << "\n1.push 2.pop 3.display 4.peep 5.change
        6.exit\nenter ur choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
                cout << "enter the element: "; cin >> ch;
                st.push(ch);
                break;
            case 2:
                st.pop();
                break;
            case 3:
                st.display();
                break;
            case 4:
                st.peep();
                break;
            case 5:
                st.change();
                break;
```


Object Oriented Programming & Data Structure

```
case 6:  
    exit(0);  
}  
}
```

Object Oriented Programming & Data Structure

Stack Application:-

There are two type of stack application.

- (1) Polish expression
- (2) Recursive

Stack machines, certain computers perform stack operation at the hardware or machine level.

Polish Expression:

When operators are written after the operands, it is called Reverse Polish Notation. There are three types of expression:

- 1) Infix
- 2) Prefix
- 3) Postfix

“Pre”, ”Post” & ”In” refer to the relative position of the operator with respect to the two operands.

Infix notation:-

- In this type of expression operators are placed between the operands.
- E.g.: $A + (B * C)$
- In this notation the order of operations to be performed is determined by the use of parenthesis or by the
- use of some operator precedence conventions.
- The levels of precedence are (highest to lowest):

Priority	Operator
1	(
2	$\wedge$, \$, $\uparrow$
3	%
4	/ or *(left to right)
5	+ or – (left to right)
6	)

Object Oriented Programming & Data Structure

Algorithm to convert infix to postfix expression

In this case, We use the stacks to convert infix to postfix. We have operator's stack, output's stack and one input string. Operator's stack works as FILO(First In Last Out). Output's stack works as FIFO (First In First Out).

The following algorithm converts infix to postfix.

- Scan input string from left to right character by character.
- If the character is an operand, put it into output stack.
- If the character is an operator and operator's stack is empty, push operator into operators' stack.
- If the operator's stack is not empty, there may be following possibilities.
 - If the precedence of scanned operator is greater than the top most operator of operator's stack, push this operator into operand's stack.
 - If the precedence of scanned operator is less than or equal to the top most operator of operator's stack, pop the operators from operand's stack untill we find a low precedence operator than the scanned character. Never pop out ('(') or (')') whatever may be the precedence level of scanned character.
 - If the character is opening round bracket ('('), push it into operator's stack.
 - If the character is closing round bracket (')'), pop out operators from operator's stack untill we find an opening bracket '(').
 - Now pop out all the remaining operators from the operator's stack and push into output stack.

Object Oriented Programming & Data Structure

Infix to postfix conversion algorithm

- 1) Scan the expression from left to right
- 2) Push the operand as they arrive in output stack
- 3) If operator arrives & operator stack is empty then push this operator onto operator stack
- 4) If incoming operator has higher precedence than the top of the operator stack then push it on to the same stack.
- 5) If incoming operator has lower precedence than the top of the operator stack then pop it and push it into output stack. Then test the incoming operator against new top of operator stack
- 6) If incoming operator has equal precedence with top of operator stack use associativity rules.
- 7) For associativity of left to right pop and push it onto output stack then push the incoming operator.
- 8) For Associativity of right to left push incoming operator on to output stack
- 9) If the scanned character is an '(', push it to the stack.
- 10) If the scanned character is an ')', pop the stack and output it until a '(' is encountered, and discard both the parenthesis
- 11) Repeat steps 2-10 until infix expression is scanned.
- 12) At the end, pop operator stack until it is not empty and push all operator into output stack.
- 13) Print the output stack.

Object Oriented Programming & Data Structure

Example

Infix: $A+(B+C*D)+E$

Postfix: $ABCD*+*E$

Step by step output for $A+(B+C*D)+E$ expression

$A + (B + C * D) + E$			
Serial No.	Symbol	Stack	Postfix Expression
1	A		A
2	*	*	A
3	(	*(	A
4	B	*(	AB
5	+	*(+	AB
6	C	*(+	ABC
7	*	*(+*	ABC
8	D	*(+*	ABCD
9	)	*(+*)	ABCD*+
10	+	*+	ABCD*+*
11	E	+	ABCD*+*E

Algorithm to convert infix to prefix expression

In this case, we have operator's stack, output's stack and one input string.

Operator's stack works as FILO(First In Last Out). Output's stack works as FIFO (First In First Out).

Infix to prefix conversion algorithm

- 1)Scan the expression from right to left
- 2)Push the operand as they arrive in output stack
- 3)If operator arrives & operator stack is empty thenpush this operator onto operator stack
- 4)If incoming operator has higher precedence than the top of the operator stack then push it on to the same stack.
- 5) If incoming operator has lower precedence than thetop of the operator stack then pop it and push it into output stack. Then test the incoming operator against new top of operator stack
- 6)If incoming operator has equal precedence with top ofoperator stack use associativity rules and pop
- 7)For associativity of left to right pop and push it onto output stack

Object Oriented Programming & Data Structure

- 9) If the scanned character is an ')', push it to the operator stack.
- 10) If the scanned character is an '(', pop and push it until a '(' is encountered, and discard both the parenthesis
- 11) Repeat steps 2-10 until infix expression is scanned.
- 12) At the end, pop operator stack until it is not empty and push all operator into output stack.
- 13) Print the output stack.

Example

Infix: (A+B*C)

Prefix : +A*BC

First of Infix expression to reverse (C*B+A)

(C*B+A)			
Serial No.	Symbol	Stack	Postfix Expression
1	(	(	
2	C	(	C
3	*	(*	C
4	B	(*	CB
5	+	(*+	CB*
6	A	(+	CB*A
7	)	(+)	CB*A+

So The Postfix Expression **CB*A+** Now reverse this Expression to get the Prefix So, Prefix is **+A*BC**

Object Oriented Programming & Data Structure

Difference between stack and Queue

STACK -

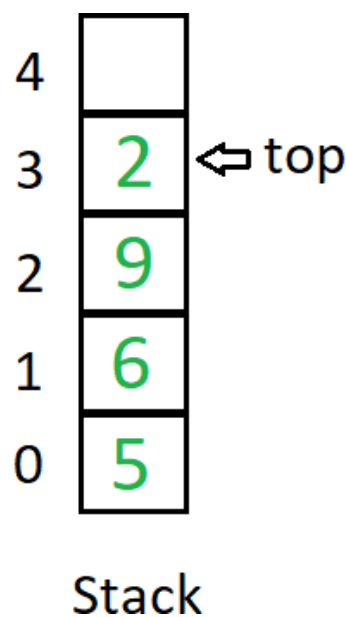
- 1.it is called as LIFO which stands for Last in first out as stack uses this property
- 2.element inserted first will be accessed last in stack as it has only one open end
- 3.recent element in stack is accessed using the value of top which points to the topmost element of stack
- 4.for inserting an element push() function is used
- 5.for deleting an element pop() function is used
- 6.example is plates placed one above other

QUEUE -

- 1.it is called as FIFO which stands for First in first out as queue uses these property
- 2.element inserted first will be served first
- 3.it has two open ends called as rear end and front end
- 4.element is deleted using pop() function from front end of queue
- 5.element is inserted using push() function from rear end of queue
- 6.example is queue for bus

Stack A is a linear data structure in which elements can be inserted and deleted only from one side of the list, called the **top**. A stack follows the **LIFO** (LastIn First Out) principle, i.e., the element inserted at the last is the first element to come out. The insertion of an element into stack is called **push** operation, and deletion of an element from the stack is called **pop** operation. In stack we always keep track of the last element present in the list with a pointer called **top**.

The diagrammatic representation of stack is given below:



Object Oriented Programming & Data Structure

Queue: A [queue](#) is a linear data structure in which elements can be inserted only from one side of the list called **rear**, and the elements can be deleted only from the other side called the **front**. The queue data structure follows the **FIFO** (First In First Out) principle, i.e. the element inserted at first in the list, is the first element to be removed from the list. The insertion of an element in a queue is called an **enqueue** operation and the deletion of an element is called a **dequeue** operation. In queue we always maintain two pointers, one pointing to the element which was inserted at the first and still present in the list with the **front** pointer and the second pointer pointing to the element inserted at the last with the **rear** pointer.

Recursion as an Application of Stack :

- A procedure that contains a procedure which calls itself is called Recursion.

OR

- Function call itself called Recursion but here 1st function call 2nd function, 2nd function call 3rd function and so on.
- Recursion is an important concept in computer science.
- Recursion is less time consuming than normal function.

Advantage of Recursion:

1. Data store in looping.
2. Recursion is a time consuming operation because of pushing all the current variables & objects onto stack.

General Algorithm for Recursion:-

Step-1: [Preface]

Save the argument & return address.

Step-2: [Body of function]

If a base criterion is completed go to step-3

Else perform partial computation & go to step-1.

Step-3: [Last Section]

Restore the most recently saved parameters & return address.

Example: - Algorithm of recursive factorial.

1. Give an Integer N. this algorithm find factorial N.
2. In this algorithm we will take a stack which is capital A. to store an activation record associated with each recursive call.
3. Each activation record contains the current value of N & the current return address whenever function is called. A copy of current argument & current address is push to the stack.
4. Whenever function completed we will fetch or pop current return address & arguments.

Algorithm:

1. [Save N & return address]
Call push(A, N, ret_add)
2. [Is the base criteria found?]
If N==0 then
Fact=1
goto step 4
Else
param=N-1
ret_add=step 3
goto step 1

Object Oriented Programming & Data Structure

3. [Calculate N!]
Fact=N*fact
4. [Restore N & ret_add]
tem_rec=pop (A) goto tem_rec

Example:

```
#include<iostream.h>
#include<conio.h>
int factorial(int n);
void main()
{
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    cout << "Factorial of " << n << " = " << factorial(n);
    getch();
}
int factorial(int n)
{
    if(n > 1)
        return n * factorial(n - 1);
    else
        return 1;
}
```